[Classic] Auto-Encoding Variational Bayes (Original VAE Paper)

- Problem
  - Assume we have a set of data: $X = \{x^{(i)}\}_{i=1}^{N}$. Assume all of the data in $X$are independent and are not from nothing; instead, it's from a continuous **latent** random var z.
  - We assume $z^{(i)}$is also from a "god-like" distribution $p_{\theta^*}(z)$, and using $z^{(i)}$we generate visible data $x^{(i)}$from $p_{\theta^*}(x|z)$.
  - Our final goal is to find a $\theta$to estimate θ^*.
  - We don't know:
    - θ^*
    - How $x^{(i)}$is related to $z^{(i)}$.
  - Intractables:
    - Probability of x:

      $p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$

      To know $p_\theta(x)$, and as $z$is continuous, we need to calculate the integral among all z's. However, $p_\theta(x|z)$can be very hard to integrate, as this can be an output of an NN, and z can be high dimensional.
      - *This formula is from $p_\theta(x, z) = p_\theta(z) \cdot p_\theta(x \mid z)$, as $z$must be selected ($p_\theta(z)$) and then given z, the probability of $x$is $p_\theta(x|z)$.To have $p_\theta(x)$, we need to consider (integrate among) all z's.*
    - True Posterior

      $p_\theta(z|x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(x)}$.

      Without knowing $p_\theta(x)$, it is impossible to calculate the true posterior.
    - Big Data: it's impossible to sample all data points in the available dataset multiple times. Need a way, like mini-batch, to update the model.
  - Goal:
    - Parameter Estimation: estimate θwithout knowing z
    - Approximate Posterior Inference: given x, get its z (encoder).
    - Marginal Inference
- The Variational Bound
  - Let:
    - $p_\theta(z|x)$be real posterior distr.
    - $q_\phi(z|x)$be the encoder
    - ϕbe the parameter of the encoder
    - θbe the decoder's parameter, and we want to let it be close to θ^*as possible.
  - **Import Expectation**: We know that $p(x)$isn't related to $z$as $p(x)$is gained by integrating over z. Plus, $p(x)$is a constant, so:

$\log p(x) = \mathbb{E}_{q_\phi(z|x)}[\log p(x)]$as a constant's expected value = that constant.

- Use bayes: $p(x) = p(x,z)/p(z|x)$
- **Introduce** q:

$\log p(x) = \mathbb{E}_q[\log \frac{p(x,z)}{p(z|x)} \cdot \frac{q(z|x)}{q(z|x)}]$ (import a "1")

- Split the log:

  $\log p(x) = \mathbb{E}_q[\log \frac{p(x,z)}{q(z|x)}] + \mathbb{E}_q[\log \frac{q(z|x)}{p(z|x)}]$

  - The first part is called **ELBO** (Evidence lower bound), and the second part is KL.
  - Why lower bound?
    - According to Jensen inequalities, KL≥ 0. Hence $\log p(x)$ is **always bigger than** $\mathbb{E}_q[\log \frac{p(x,z)}{q(z|x)}]$.
    - We have no idea how to determine $\mathbb{E}_q[\log \frac{q(z|x)}{p(z|x)}]$as $p(z|x)$is unknown. We know everything about ELBO: we know $p_\theta(x,z)$*(Yes, we do. We know this but it's hard to run integral.)* Thus, we just need to maximize the ELBO (so $\log p(x)$will be even larger) to maximize the **reality** $p(x)$. ($p(x)$happened, so it is a GROUND TRUTH)
- Two forms of objective functions:
  - $\mathcal{L}(\theta, \phi; x^{(i)}) = \mathbb{E}_{q_\phi(z|x)}[-\log q_\phi(z|x) + \log p_\theta(x,z)]$(directly from breaking down log)
  - $$\begin{aligned}\mathcal{L} &= \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(z)p_\theta(x|z)}{q_\phi(z|x)}\right] \\ &= \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x|z) + \log p_\theta(z) - \log q_\phi(z|x)\right] \\ &= \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] + \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(z) - \log q_\phi(z|x)\right] \\ &= \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] + \mathbb{E}_{q_\phi(z|x)}\left[-\left(\log q_\phi(z|x) - \log p_\theta(z)\right)\right] \\ &= \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{q_\phi(z|x)}{p_\theta(z)}\right] \\ &= \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{Reconstruction}} - \underbrace{D_{KL}(q_\phi(z|x)||p_\theta(z))}_{\text{Regularizer}}\end{aligned}$$
    - Used more in implementation
    - Reconstruction: given z, which is from encoder q_ϕ, the probability that the decoder believes that $x$can be recovered.
    - KL: we don't want $q_\phi$to diverge too much; we want it to be close to the preterior distribution $p_\theta(z)$ (As $\theta$is our parameter, we can make $p_\theta$be normal distribution).

- Optimizer: The SGVB Estimator

  $$\underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{Reconstruction}} - \underbrace{D_{KL}(q_\phi(z|x)||p_\theta(z))}_{\text{Regularizer}}$$

  - We want to find the gradient of ϕ. However, the distribution, q_ϕ, contains ϕitself. We cannot move the gradient into the expectation, either.
  - One classic way to do this is by Naive Monte Carlo gradient estimator:

    $\nabla_\phi \mathbb{E}_{q_\phi}[f(z)] \simeq \frac{1}{L}\sum f(z)\nabla_\phi \log q_\phi(z)$

    - However, the variance is huge -> the training is not stable.
    - Why? (By Gemini)
  - Instead of directly sampling from q_ϕ, we make:

$\tilde{z} = g_\phi(\epsilon, x) \quad \text{where} \quad \epsilon \sim p(\epsilon)$

- $\epsilon$is an independent noise var to simulate the sampling randomness.
- $g_\phi(\cdot)$is a deterministic differentiable function, such as an NN.

- Re-write the loss function:

  $$\tilde{\mathcal{L}}^B(\theta, \phi; x^{(i)}) = \underbrace{-D_{KL}(q_\phi(z|x^{(i)})||p_\theta(z))}_{\text{directly calculate; no sampling}} + \underbrace{\frac{1}{L}\sum_{l=1}^{L} \log p_\theta(x^{(i)}|z^{(i,l)})}_{\text{sample the encoder distribution ONLY}}$$

  where

  $z^{(i,l)} = g_\phi(\epsilon^{(l)}, x^{(i)})$

- Auto-Encoding Variational Bayes (to handle big-data problems)

  $\mathcal{L}(\theta, \phi; X) \simeq \frac{N}{M}\sum_{i=1}^{M} \tilde{\mathcal{L}}(\theta, \phi; x^{(i)})$, where

  N: size of the total dataset

  M: minibatch size

  Findings: if M is large enough, for each data point, only one sampling is needed. No need for multiple samplings, and thus it decreases the time cost.

  Algorithm 1 Minibatch version of the Auto-Encoding VB (AEVB) algorithm. Either of the two SGVB estimators in section 2.3 can be used. We use settings $M = 100$ and $L = 1$ in experiments.

  $\boldsymbol{\theta}, \boldsymbol{\phi} \leftarrow$ Initialize parameters
  repeat
  $\mathbf{X}^M \leftarrow$ Random minibatch of M datapoints (drawn from full dataset)
  $\boldsymbol{\epsilon} \leftarrow$ Random samples from noise distribution $p(\boldsymbol{\epsilon})$
  $\mathbf{g} \leftarrow \nabla_{\boldsymbol{\theta},\boldsymbol{\phi}} \widetilde{\mathcal{L}}^M(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{X}^M, \boldsymbol{\epsilon})$ (Gradients of minibatch estimator (8))
  $\boldsymbol{\theta}, \boldsymbol{\phi} \leftarrow$ Update parameters using gradients $\mathbf{g}$ (e.g. SGD or Adagrad [DHS10])
  until convergence of parameters $(\boldsymbol{\theta}, \boldsymbol{\phi})$
  return $\boldsymbol{\theta}, \boldsymbol{\phi}$

- Example: VAE
  - Prior $p_\theta(z)$: Centered isotropic multivariate Gaussian

    $p(z) = \mathcal{N}(z; 0, I)$, frozen, no training needed.

    - Practically, this makes sure the encoder is continuous, dense, and smooth, or, the *manifold*, *(just like performing interpolation on a manifold*). That is to say, as $p(z)$is in the KL, this makes the encoder be as close to the standard normal distribution as possible.
    - Without the constraint from the KL, the encoder will degrade to a normal, discrete encoder.
  - Encoder $q_\phi(z|x)$: Multivariate Gaussian distribution with a diagonal covariance matrix.

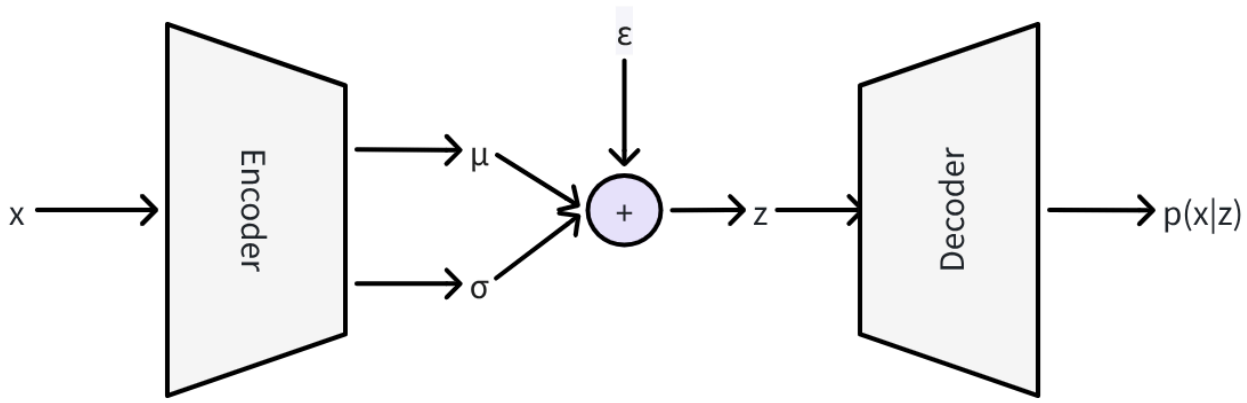
    $\log q_\phi(z|x^{(i)}) = \log \mathcal{N}(z; \mu^{(i)}, \sigma^{2(i)} I)$

    - Input: x
    - Output: $\mu^{(i)}$and $\sigma^{(i)}$ instead of z.
    - Reparameterization:
      - $z^{(i,l)} = \mu^{(i)} + \sigma^{(i)} \odot \epsilon^{(l)} \quad \text{where} \quad \epsilon^{(l)} \sim \mathcal{N}(0, I)$, and $\odot$means element-wise multiplication.
      - Here we calculate the z
  - Decoder $p_\theta(x|z)$:

        - For continuous numerical data (e.g., facial photos): Use a multivariate Gaussian distribution.
        - For binary data (e.g., MNIST): Use a Bernoulli distribution.
            - Input: $z^{(i,l)}$
            - Output: $p_\theta(x|z)$
    - Bring everything back to the loss:

$$\mathcal{L}(\theta, \phi; x^{(i)}) \simeq \underbrace{\frac{1}{2}\sum_{j=1}^{J}(1 + \log((\sigma_j^{(i)})^2) - (\mu_j^{(i)})^2 - (\sigma_j^{(i)})^2)}_{\text{KL Divergence Term}} + \underbrace{\frac{1}{L}\sum_{l=1}^{L}\log p_\theta(x^{(i)}|z^{(i,l)})}_{\text{Reconstruction Term}}$$

        - We want to maximize this, theoretically.
        - For KL term:
            - $-\mu^2$: we want it to be as close to 0 as possible
            - $-\sigma^2 + \log(\sigma^2)$: we want the std/variance to close to 1
            - This will try to make the encoder close to $\mathcal{N}(z; 0, I)$
    - For Reconstruction Term, it can be binary cross entropy or MSE, based on the tasks.



- Why do we want to use VAE?
    - Normal AE has dead-zones. For example, $z_{x=7} = (-1, -1)$, and $z_{x=8} = (2, 2)$, then what is x when $z = (-1.5, -1.5)$? For normal AE, this zone can be meaningless.
    - But because the **latent space learned by VAE** is dense, continuous, and smooth, the zone between (-1, -1) and (2, 2) is also meaningful, representing a **semantic interpolation** (or morphing) between '7' and '8'
    - In some ways, it's better than GAN: easier to train (non-adversal) and more diverse.
    - It can also detect anomalies, just like normal AEs do.
- Work Cited
    - Paper
    - Chat History